

Week 5 - Lecture 9 - Problem: Projectile Simulation

Background

A projectile has properties such as an identifier, size, energy, and position. A `Projectile` object can store these values and provide methods to inspect or change its state. Its `fire` method will update its position repeatedly, while a separate helper function will display each position in the terminal.

The simulation uses gravity and a deliberately simplified horizontal drag model. In this exercise, `AERO_DRAG` represents a constant horizontal deceleration; it is not a realistic aerodynamic drag coefficient. The goal is to practice classes, state changes, loops, and using a helper function, rather than build a precise physics model.

The provided `animate(x, y)` function is an interface: call it with the projectile's current coordinates to display a frame. You do not need to understand how the character grid or terminal clearing works.

Task

Part 1: Define simulation constants

Define module-level constants for gravity and horizontal deceleration. For example:

```
G = 9.81
AERO_DRAG = 0.4
```

Use a small time step such as `dt = 0.02` seconds in the simulation.

Part 2: Create the `Projectile` class

Implement a `Projectile` class whose initializer stores:

```
id, size, energy, x_pos, y_pos
```

Add these methods:

- `get_data()` returns a tuple containing `id`, `size`, and `energy`.
- `get_position()` returns the current position as `(x_pos, y_pos)`.
- `fire(velocity, theta)` simulates the projectile's flight. The angle `theta` is in radians.

Inside `fire`, calculate the initial velocity components with `math.cos` and `math.sin`. While the projectile is at or above ground level:

1. Call `animate(self.x_pos, self.y_pos)` to display its current position.
2. Update its position using the current velocity and time step.
3. Update vertical velocity using gravity and horizontal velocity using the simplified deceleration.

Use this update model for the exercise:

```

self.x_pos += x_vel * dt
self.y_pos += y_vel * dt
y_vel -= G * dt
x_vel -= AERO_DRAG * dt

```

Part 3: Animate the projectile

Use the following helper function. It requires the `time` module. Place it outside the `Projectile` class and call it from `fire`.

```

import time

def animate(x, y):
    width, height = 60, 30
    scale = 100
    screen = [[" " for _ in range(width)] for _ in range(height)]

    column = round(x * scale)
    row = height - 1 - round(y * scale)
    if 0 <= column < width and 0 <= row < height:
        screen[row][column] = "*"

    print("\033[2J\033[H", end="")
    print("\n".join("".join(line) for line in screen), flush=True)
    time.sleep(0.1)

```

Part 4: Create and fire a projectile

Create a projectile at `(0, 0)` and fire it with a chosen speed and launch angle. For example, convert 45 degrees to radians using `math.radians(45)`. Observe how the `*` moves in the terminal.

Requirements

- Store each projectile's data and position in instance attributes.
- Implement and use the three methods `get_data`, `get_position`, and `fire`.
- Use `math.cos`, `math.sin`, and an angle in radians to calculate launch velocity components.
- Use a loop and a time step to update position and velocity until the projectile reaches the ground.
- Call the provided `animate(x, y)` helper once per simulation step.
- Use `G` and `AERO_DRAG` as module-level constants.
- Do not use a graphics library.

Discussion Questions

1. Which values describe an individual projectile, and which values are simulation constants?
2. How does `fire` change the state of the projectile?
3. Why is `animate` a standalone helper function rather than a method of `Projectile`?
4. How do launch speed and launch angle affect the path?

5. Why is this horizontal drag model simplified, and what would a more realistic model need to consider?

Extension Challenges

1. Add a **Cannon** class that creates or stores a projectile and fires it. Describe the association between the cannon and projectile.
2. Allow the launch speed and angle to be entered by the user.
3. Change the animation scale or grid dimensions and observe how the displayed trajectory changes.